CAPSTONE PROJECT REPORT

on

LUNAR CRATER DETECTION AND SHAPEFILE GENERATION USING A SEMI-SUPERVISED YOLOv8 FRAMEWORK

Submitted by

Anshit Sanjay (1032210284)

Anushka Mankar (1032210272)

Nityashree Kumar (1032211414)

Under the Guidance of

Dr. Apurva A Naik

Department of Electrical and Electronics Engineering

Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY, PUNE
[2021 - 2025]

DECLARATION

We the undersigned, declare that the work carried under Capstone Project entitled **"Lunar Crater Detection and Shapefile Generation Using a Semi-Supervised YOLOv8 Framework"** represents our idea in our own words. We have adequately cited and referenced the original sources where other ideas or words have been included. We also declare that We have adhered to all principles of academic honesty and integrity and have not misprinted or fabricated or falsified any ideas/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the University and can also evoke penal action from the sources which have thus not been properly cited or whom proper permission has not been taken when needed.

PRN Number	Name of student	Signature with date
1032210284	Anshit Sanjay	
1032210272	Anushka Mankar	
1032211414	Nityashree Kumar	

Dr. Apurva A. Naik	Dr. Anjali Ashkhedkar	Dr. Parul Jadhav
Project Guide	Project Coordinator	Program Director

Date:

Place: Pune

ACKNOWLEDGEMENT

We would like to extend our heartfelt gratitude and appreciation to all those who have contributed to the successful completion of the “Lunar Crater Detection and Shapefile Generation Using a Semi-Supervised YOLOv8 Framework” project. This endeavor would not have been possible without the dedication, support, and expertise of several individuals, and we would like to acknowledge their invaluable contributions.

First and foremost, we express our sincere thanks to our project team, whose hard work and tireless efforts have made this project a reality. Each team member brought their unique skills and insights, which played a pivotal role in the project's success. Their commitment and collaboration were instrumental in achieving our goals.

We owe deep gratitude to our Guide Dr. Apurva. A. Naik and Project Coordinator Dr. Anjali Ashkhedkar. They provided us with valuable guidance, constant encouragement, and unwavering support throughout the project. Their mentorship inspired us to overcome challenges and transform our initial concept into a successful and well-executed project.

We would also like to extend our sincere thanks to the Department of Electronics and Communication Engineering for providing the resources and academic environment necessary for the completion of our work.

Finally, we would like to express our thanks to all those who have helped us directly and indirectly in the successful completion of this project.

Table of Contents

ABSTRACT

This report presents a modular and semi-automated pipeline for the detection and geospatial mapping of lunar craters using high-resolution calibrated imagery from the Orbiter High Resolution Camera (OHRC) onboard India's Chandrayaan-2 mission. Leveraging a semi-supervised annotation strategy, an initial YOLOv8 model was trained on 576 publicly labeled crater images and then used to auto-annotate over 3,600 OHRC tiles. The model was fine-tuned on this expanded dataset, achieving 88.9 % precision, 87.7 % recall, and a mAP@0.5 of 94.99 % after 50 epochs. The end-to-end workflow includes: (1) converting calibrated .img files (with accompanying .xml metadata) to GeoTIFFs with GDAL and applying GCPs; (2) tiling and preserving affine transforms with Rasterio; (3) semi-supervised annotation and YOLOv8 model training on Google Colab; (4) inference on macOS M4; and (5) mapping pixel-level detections back to selenographic coordinates and exporting shapefiles via GeoPandas. Key challenges included handling tile edge artifacts, maintaining geospatial metadata integrity, and managing annotation noise. The final system produces accurate crater shapefiles suitable for lunar geological analysis and lays the groundwork for future enhancements such as multi-class detection, real-time deployment, and context-aware tiling.

LIST OF TABLES

LIST OF FIGURES

ABBREVIATIONS

AI	Artificial Intelligence
DL	Deep Learning
ML	Machine Learning
CNN	Convolutional Neural Network
YOLO	You Only Look Once
OHRC	Orbiter High Resolution Camera
GCP	Ground Control Point
GDAL	Geospatial Data Abstraction Library
CRS	Coordinate Reference System
GeoTIFF	Georeferenced Tagged Image File Format
PNG	Portable Network Graphics
mAP	mean Average Precision
F1-Score	Harmonic Mean of Precision and Recall
CSV	Comma-Separated Values
XML	eXtensible Markup Language
GPU	Graphics Processing Unit
PR Curve	Precision-Recall Curve
R Curve	Recall Curve
P Curve	Precision Curve

CHAPTER 1
INTRODUCTION

1.1 Background

The Moon, Earth's only natural satellite, has been a subject of immense scientific interest for decades. It serves as a crucial reference point for understanding planetary processes, early solar system history, and the geological evolution of terrestrial bodies. One of the most prominent and studied features of the lunar surface is the impact crater. These craters are formed when meteoroids, asteroids, or comets collide with the lunar surface, resulting in circular depressions of various sizes. Unlike Earth, the Moon lacks an atmosphere, plate tectonics, and significant weathering processes, allowing these features to remain well-preserved for millions or even billions of years.

Crater analysis plays a vital role in planetary geology. The distribution, size, shape, and density of craters are used to estimate the relative and absolute ages of lunar regions. Older surfaces typically exhibit higher crater densities, while younger surfaces have fewer impact marks. Furthermore, crater morphology can offer insights into the subsurface composition, mechanical properties of lunar materials, and the dynamics of impact events. Hence, accurate crater detection and mapping are fundamental for lunar stratigraphy, chronology, and mission planning.

Over the years, numerous lunar missions have provided valuable imagery for scientific analysis. Among them, ISRO's Chandrayaan-2 mission has contributed significantly to lunar exploration through its suite of advanced payloads, particularly the Orbiter High Resolution Camera (OHRC). The OHRC captures panchromatic imagery at an exceptionally fine spatial resolution of 0.25 meters per pixel, enabling the observation of minute surface features. This level of detail is ideal for identifying micro-craters, boulder fields, and fine geomorphological variations that are otherwise undetectable in lower-resolution datasets.

Despite the availability of such high-quality imagery, the task of manually identifying and cataloging craters remains tedious, time-consuming, and prone to subjectivity. As the volume of high-resolution satellite data increases, traditional methods of visual interpretation become infeasible for large-scale analysis. The need for automated and scalable solutions is more pressing than ever.

In recent years, deep learning—a subfield of artificial intelligence—has emerged as a powerful tool for automated image analysis. Convolutional Neural Networks (CNNs), in particular, have revolutionized the fields of computer vision and remote sensing by demonstrating high accuracy in tasks such as object detection, classification, and semantic segmentation. These models are capable of learning hierarchical feature representations directly from raw image data, making them particularly well-suited for detecting complex patterns like craters in lunar imagery.

By leveraging deep learning-based object detection algorithms, this project proposes a comprehensive framework for automated crater detection using OHRC imagery. The objective is to reduce the dependency on manual labor, enhance the consistency of crater identification, and provide geospatially accurate outputs for further scientific and operational applications.

1.2 Problem Statement

The manual identification and annotation of lunar craters in high-resolution satellite imagery is not only time-consuming but also impractical at scale. Traditional image processing techniques, while useful in some cases, often lack robustness in dealing with varied crater morphologies, illumination conditions, and surface textures. Furthermore, existing automated methods are often not tailored to work with ultra-high-resolution data like that produced by OHRC, limiting their effectiveness.

There is a critical need for a fully automated, efficient, and geospatially-aware system that can accurately detect and map craters across large lunar regions using high-resolution imagery. Such a system must handle diverse crater sizes, overlapping structures, and irregular terrains while producing outputs suitable for scientific analysis and integration with Geographic Information Systems (GIS).

1.3 Objectives

This project is centered on developing a robust and modular deep learning pipeline for crater detection and mapping using OHRC imagery. The key objectives are:

- **To preprocess OHRC satellite imagery** and convert it into georeferenced formats suitable for deep learning-based analysis.
- **To design and train a deep learning model**, particularly using object detection architectures, for automated recognition of craters across varying terrains.
- **To implement an efficient annotation strategy** and labeling process for training the model on representative lunar regions.
- **To map the detected craters to selenographic coordinates**, ensuring the spatial accuracy of the outputs.
- **To generate GIS-compatible outputs**, including shapefiles and metadata, for integration with lunar maps and scientific databases.
- **To create a reproducible, extendable pipeline** that can be applied to other lunar datasets and regions beyond the current scope.

Table 1.1: Comparison Between Traditional Methods and Deep Learning for Crater Detection

Aspect	Traditional Image Processing	Deep Learning-Based Approach
Accuracy	Moderate; dependent on feature engineering	High; learns features automatically
Scalability	Low; requires manual tweaking per dataset	High; adaptable with large datasets
Robustness to Variability	Limited; sensitive to lighting and textures	Strong; can generalize over diverse inputs
Automation Level	Low to medium	High
Annotation Requirement	Low	High (for training data)
Interpretability	Higher; based on defined rules	Lower; model functions as a black box
Computation Requirement	Low	High
Suitability for OHRC Data	Limited due to resolution and complexity	Ideal due to learning capability

1.4 Scope of the Project

The scope of this project is specifically focused on 2D detection and geospatial mapping of lunar impact craters using high-resolution imagery captured by OHRC onboard Chandrayaan-2. It involves the full lifecycle of data handling—from image preprocessing, annotation, and model training to post-inference spatial mapping and output generation.

The project does **not** cover:

- The detection of other lunar features such as ridges, rilles, domes, or boulders.
- 3D surface reconstruction or elevation modeling.
- Real-time onboard crater detection or hardware deployment.

However, the pipeline is designed to be modular, allowing easy adaptation to other datasets (such as LRO imagery) or the addition of new features in future iterations.

CHAPTER 2

LITERATURE REVIEW

The detection and analysis of craters on planetary bodies, particularly the Moon, has long been a topic of significant interest in planetary science. Over the years, various techniques have been developed to identify, classify, and study impact craters using high-resolution imagery. The advent of machine learning, particularly deep learning, has significantly transformed this field, providing new methods for automatic crater detection. This chapter reviews the evolution of these methodologies, focusing on traditional image processing, machine learning techniques, and deep learning approaches, with a special emphasis on datasets like those obtained from Chandrayaan-2's OHRC. The chapter also identifies the strengths and limitations of existing techniques, offering the motivation for developing a deep learning-based pipeline for OHRC data.

2.1 Traditional Methods for Crater Detection

In the early stages of crater detection, manual annotation was the predominant method, involving planetary scientists painstakingly identifying craters in optical and photographic data. While highly accurate, this approach was inherently time-consuming, subjective, and unsustainable for large datasets. As digital imagery became more accessible, automated and semi-automated image processing techniques were introduced.

Common traditional techniques included:

- **Edge Detection** (e.g., Sobel, Canny): Used to highlight the boundaries of circular craters by detecting significant changes in pixel intensity.
- **Hough Circle Transform (HCT)**: A widely used technique for detecting circular or elliptical shapes, particularly suited for simple crater geometries.
- **Thresholding and Morphological Operations**: Applied to segment images based on pixel intensity and shape, isolating potential crater features.

While these methods were effective for high-contrast images with clear crater boundaries, their performance diminished when applied to real-world lunar imagery, due to several challenges:

- **Lighting Variability**: Lunar imagery often includes varying lighting conditions due to the Sun's angle, which complicates crater detection.

- **Degraded or Eroded Craters**: Older or weathered craters may have indistinct boundaries, making detection harder.
- **Overlapping Features**: In complex terrains, craters often overlap, making them difficult to detect individually.
- **High False Positive Rates**: Features such as domes or pits can closely resemble craters, resulting in false positives.

Thus, while traditional methods established the groundwork for crater detection, they were limited in scalability, generalizability, and robustness, requiring significant post-processing or expert validation.

2.2 Machine Learning Approaches

In response to the limitations of traditional methods, machine learning (ML) techniques began to gain traction for crater detection. ML algorithms, which learn patterns from data, offered a more flexible and robust alternative to rule-based approaches, enabling better generalization across varied terrains.

Prominent ML techniques for crater detection included:

- **Support Vector Machines (SVMs) and Random Forests**: These models were trained on handcrafted features such as gradient histograms, texture descriptors, and shape features. While they provided better performance than traditional methods, they still required manual feature engineering and were sensitive to dataset-specific characteristics.
- **Template Matching**: A method where predefined crater templates were matched against regions of interest in an image to identify potential craters.

Despite improving detection accuracy, these ML methods had several limitations. Most notably, they depended on **feature engineering**, which is manual and highly specific to the dataset. This method cannot capture complex spatial hierarchies inherent in high-resolution lunar imagery. Furthermore, these techniques struggled to scale effectively when applied to high-resolution datasets like OHRC images due to increased noise and intra-class variation in crater appearances.

2.3 Deep Learning for Crater Detection

The advent of deep learning, particularly **Convolutional Neural Networks (CNNs)**, marked a transformative shift in crater detection. Unlike traditional ML models, CNNs automatically learn relevant features from raw pixel data, significantly reducing the need for manual feature engineering and enabling more accurate and scalable crater detection.

Key advancements in deep learning for crater detection include:

2.3.1 Classification and Detection Models

- **CNN Classifiers**: Early applications used CNNs for binary classification (crater vs. non-crater) on image patches. While promising, this approach faced limitations in handling spatial relationships and detecting overlapping craters.
- **Object Detection Frameworks**: More recent efforts have adopted advanced region-based detection models, such as:
    - **Faster R-CNN**: A two-stage approach that generates region proposals followed by object classification and localization.
    - **YOLO (You Only Look Once)**: A single-stage detector that simultaneously detects multiple objects in a single pass, offering high speed and efficiency, which is ideal for large-scale datasets like lunar imagery.
    - **SSD (Single Shot Multibox Detector)**: A real-time detection model that balances speed and accuracy, suitable for detecting craters in large-scale images.

These object detection models are capable of detecting and localizing multiple craters in a single pass, making them highly efficient for processing large lunar datasets.

2.3.2 Semantic Segmentation

In parallel, some researchers have explored **semantic segmentation** for crater detection using models like **U-Net** and **DeepLab**. These architectures perform pixel-wise segmentation, providing more detailed shape extraction for craters. However, these models require dense annotations and are computationally expensive, which can be a limitation for large-scale datasets.

2.3.3 Challenges with Deep Learning

While deep learning methods have revolutionized crater detection, they are not without challenges:

- **Imbalanced Datasets**: Lunar datasets often have an imbalance between craters and non-crater background features, leading to difficulties in model training and performance.
- **Annotation Bottlenecks**: High-quality labeled datasets are essential for training deep learning models, but manual annotation is time-consuming and resource-intensive.
- **Overfitting**: Deep learning models may overfit to specific terrains, lighting conditions, or resolutions if not properly regularized or diversified.
- **Small or Degraded Craters**: Detecting small craters or craters that are degraded or shadowed remains a challenge, particularly in noisy regions or areas with low contrast.

2.4 OHRC Imagery and Related Datasets

The **Orbiter High Resolution Camera (OHRC)** aboard the **Chandrayaan-2** mission is a state-of-the-art imaging instrument that captures lunar surface images at 0.25 m spatial resolution. This high-resolution imagery allows for detailed crater detection and the study of fine surface features, including micro-craters and their morphological characteristics.

Although several crater detection studies have used data from missions like:

- Lunar Reconnaissance Orbiter (LRO)
- Clementine
- Kaguya (SELENE)
- Apollo missions

Few studies have fully leveraged OHRC data, primarily due to its limited availability and the technical complexity involved in processing ultra-high-resolution images. This creates an opportunity to develop specialized deep learning pipelines specifically designed for OHRC data, which will address challenges such as:

- Fine-scale feature differentiation.
- High crater density in specific regions

CHAPTER 3

SYSTEM DESIGN AND TOOLS USED

The system design for this project establishes a highly modular, intelligent, and scientifically driven workflow for detecting lunar craters using calibrated OHRC (Orbital High-Resolution Camera) imagery. At its core, this system seeks to transform complex satellite data into meaningful geological insights by integrating advanced geospatial processing with deep learning. Unlike traditional image-based methods that may lose spatial fidelity, our design prioritizes georeferenced, high-precision data from the very beginning, enabling downstream applications like lunar mapping, geological surveying, and cross-referencing with historical crater datasets.

This architecture supports not only crater detection but also the retention of precise spatial metadata—ensuring that every prediction is tied back to the Moon's selenographic coordinate system. Such accuracy is vital for researchers studying crater distributions, impact patterns, or planetary formation. Moreover, our pipeline is scalable, reproducible, and modular—allowing easy integration with future upgrades in machine learning models, annotation tools, or geospatial standards. Each component, from data ingestion to shapefile generation, has been chosen for its scientific robustness and compatibility with global GIS ecosystems. Additionally, by embracing modern paradigms like semi-supervised learning and coordinate transformation, we establish a method that is both technologically advanced and scientifically relevant.

3.1 System Architecture Overview

The architecture of our pipeline is composed of six key stages, forming a seamless progression from raw satellite data to geospatially accurate crater detections. These stages ensure that each image tile retains its spatial context, allowing for accurate scientific analysis. Importantly, the design upholds flexibility—new modules or improvements can be plugged in without rewriting the entire system. Below is an expanded explanation of each stage

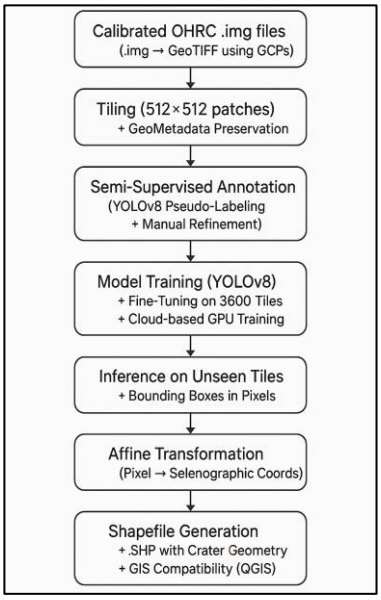


Figure 3.1: Flow-chart representing the modular architecture of the crater detection system

1. **Image Preprocessing & Georeferencing**: Calibrated OHRC .img files are obtained from planetary data archives such as ISRO's Chandrayaan-2 mission or NASA's Planetary Data System. These images initially lack geospatial referencing, which makes them unsuitable for direct scientific use. To address this, we use GDAL to convert .img files to GeoTIFF format—a standardized raster format that supports spatial metadata embedding. Georeferencing is the process of assigning real-world coordinates to each pixel in the image using Ground Control Points (GCPs). These GCPs are derived from known lunar landmarks or spacecraft metadata. The image is then projected into a standard lunar projection such as Simple Cylindrical. This transformation ensures that subsequent tiles and detections are spatially aligned, which is critical for accurate crater cataloguing.

2. **Tiling and Dataset Preparation**: OHRC images are extremely high-resolution and cannot be fed into machine learning models directly due to memory constraints. Therefore, the images are broken down into 512×512 pixel tiles using GDAL and custom Python scripts. Each tile is generated while preserving its geospatial metadata, including an affine transformation matrix that describes the spatial relationship between image pixels and real-world coordinates. This metadata is crucial during the post-processing phase. Tiles are converted to PNG format to be compatible with computer vision frameworks and are organized into structured datasets for annotation

and training. This step allows for scalable model training and ensures efficient GPU memory usage.

3. **Semi-supervised Annotation Strategy**: Annotating thousands of crater instances manually is time-consuming and labor-intensive. To overcome this, we implemented a semi-supervised annotation approach. We began by training a YOLOv8 model on a small publicly available crater dataset. This base model was then used to generate pseudo-labels on our OHRC tiles. These automatically generated labels were reviewed and refined by human annotators using annotation tools. This hybrid process reduces manual effort while improving annotation accuracy. The phrase **"semi-supervised selenographic inference"** describes this strategy, combining AI-driven annotation with spatial accuracy grounded in lunar coordinates. The word "selenographic" refers to the coordinate system used for mapping features on the Moon, analogous to geographic coordinates on Earth. It ensures that annotations are not just image-based but geospatially meaningful, enabling crater positions to be referenced accurately on lunar maps.

4. **Model Training and Fine-tuning (YOLOv8)**: YOLOv8, a state-of-the-art object detection model, was selected for its speed and accuracy. We fine-tuned it on our pseudo-labeled dataset of over 3600 tiles. During training, the model learned to identify and localize crater features such as rims and shadows across varying lighting and terrain conditions. Hyperparameters including learning rate, batch size, and confidence thresholds were carefully tuned using validation metrics like mean Average Precision (mAP), Intersection over Union (IoU), and recall. Training was executed on cloud GPU instances to expedite the process. The model's architecture, with its anchor-free prediction heads and streamlined backbone, proved highly effective in detecting small and irregular crater shapes.

5. **Inference and Bounding Box Mapping**: Once trained, the model was applied to unseen image tiles for inference. Each prediction outputs a bounding box in pixel coordinates, indicating the location of detected craters. To convert these pixel-based detections into lunar coordinates, we used the affine transformation matrices stored during tiling. This step—known as spatial transformation—allows us to accurately position each crater detection within the Moon’s selenographic coordinate grid. Such

spatial fidelity is essential for scientific analyses that require correlation with existing lunar maps or geological layers.

6. **Shapefile Generation with Selenographic Coordinates**: The final detections are transformed into vector geometries and saved as shapefiles (.shp) using the Shapely and Fiona libraries. These shapefiles include metadata such as crater size, bounding box coordinates, confidence scores, and tile origin. The use of selenographic coordinates ensures that these files can be loaded directly into GIS platforms like QGIS, enabling users to visualize crater distributions, overlay results with elevation maps, or perform further spatial analysis. This stage effectively bridges the gap between machine learning outputs and traditional planetary science workflows

Table 2.1: Summary of Existing Crater Detection Approaches

Study	Year	Approach	Dataset Used	Strengths	Limitations
Kim et al.	2015	Hough Transform + Edge Filters	LOLA DEM	Simple, interpretable	Poor on overlapping craters, low recall
Silburt et al.	2019	CNN (Convolutional Neural Net)	LROC NAC	Strong at feature learning	Requires extensive manual annotation
Cheng et al.	2020	U-Net for crater segmentation	DEM + LROC	Pixel-level accuracy	Computationally expensive
Mikhail et al.	2022	Transfer Learning (ResNet)	Lunar Recon Datasets	Leverages pre-trained weights	Generalization on small craters limited
This Work (Proposed)	**2025**	YOLOv8 + Semi-supervised Pseudo-labelling	OHRC (Chandrayaan-2)	Fast inference, spatially precise, scalable	Requires careful coordinate transformation

3.2 Tools and Technologies Used

3.2.1 Programming Language: Python

Python is a versatile and widely used programming language, especially in the fields of scientific computing, machine learning, and geospatial analysis. It was chosen as the foundation of our system for its extensive ecosystem, active community, and compatibility with numerous powerful libraries. Python's simplicity and flexibility make it an excellent choice for rapid prototyping, testing, and developing complex data processing pipelines. It allowed us to integrate custom scripts with various machine learning models, geospatial tools, and data visualization libraries seamlessly.

Key Python libraries used include:

- **NumPy** for efficient numerical operations on arrays and matrices.
- **Pandas** for handling and manipulating tabular data, especially for managing metadata related to crater detections.
- **OpenCV** for image preprocessing and visualization.
- **TensorFlow/PyTorch** (if applicable in the context of training) for model optimization and deployment.
- **GDAL** for geospatial data handling.

3.2.2 Geospatial Tools

Geospatial tools are critical for transforming raw satellite data into usable spatial information. These tools enable precise manipulation of geographic data, ensuring that all stages of our pipeline maintain accurate real-world geographic context.

- **GDAL (Geospatial Data Abstraction Library)**: GDAL is a low-level library that provides comprehensive functionality for reading, writing, and transforming raster and vector data formats. In our system, GDAL was primarily used for converting raw OHRC imagery in .img format into georeferenced GeoTIFF files. GeoTIFF is a common raster data format that supports embedding geospatial metadata such as projection information, coordinate systems, and pixel resolutions. GDAL also facilitated spatial transformations during georeferencing, such as assigning Ground

Control Points (GCPs) and transforming images into a standardized lunar projection system.

- Key Features of GDAL:
  - Supports a wide range of raster and vector formats, making it flexible for various geospatial datasets.
  - Georeferencing and reprojection capabilities, essential for aligning imagery with global coordinate systems.
  - Spatial analysis functions for calculating distances, areas, and other geographical metrics.

- **Shapely & Fiona**:
  - **Shapely** is a Python library used for manipulation and analysis of geometric objects. In our workflow, Shapely was critical for transforming the pixel-based bounding box outputs from the YOLO model into geometrically accurate polygon shapes that represent crater boundaries. This allows us to visualize the detected craters as vector geometries.
  - **Fiona** is used alongside Shapely to write the generated geometric objects (like polygons) into shapefile format (.shp). Shapefiles are the industry standard for spatial data exchange and are compatible with most Geographic Information Systems (GIS) tools, such as QGIS and ArcGIS. Fiona provides the interface for reading and writing these shapefiles.

- **Pyproj**: Pyproj is a Python interface to the PROJ library, which is used for transforming between different geographic coordinate systems. In the context of our system, Pyproj allowed us to convert pixel-based image coordinates into lunar selenographic coordinates. This transformation ensures that crater locations identified by the model align with real-world coordinates on the Moon's surface, crucial for scientific analyses.

3.2.3 Deep Learning Frameworks

Deep learning models have become a cornerstone of modern image analysis tasks, especially for detecting and classifying complex objects like lunar craters. The deep learning frameworks

we selected were chosen based on their speed, ease of use, and state-of-the-art performance in object detection tasks.

- **YOLOv8 (Ultralytics)**: YOLO (You Only Look Once) is an object detection model renowned for its speed and accuracy. YOLOv8, the latest version in the series, is particularly well-suited for our task of detecting lunar craters due to its lightweight architecture and ability to perform real-time inference. YOLOv8 was trained on both publicly available crater datasets and pseudo-labeled OHRC image tiles to detect craters across various lighting and terrain conditions. YOLO's anchor-free prediction heads make it well-suited to detect irregularly shaped objects like craters.
    - Key Features of YOLOv8:
        - High inference speed, enabling the processing of large datasets quickly.
        - High accuracy, making it ideal for precise crater localization.
        - Ease of integration with custom datasets, allowing for effective fine-tuning.
- **OpenCV**: OpenCV (Open Source Computer Vision Library) is an open-source computer vision and machine learning software library. We used OpenCV for several tasks, including image preprocessing (e.g., format conversion, resizing, and visualizing model predictions). OpenCV also helped in drawing bounding boxes around detected craters for visual inspection and validation. Its powerful functions for image manipulation made it indispensable for efficiently handling visual data during both preprocessing and inference stages.
    - Key Features of OpenCV:
        - Image manipulation and transformation (e.g., scaling, color adjustments).
        - Real-time computer vision, essential for visualizing detections.
        - Drawing and overlaying geometric shapes like bounding boxes for model outputs.
- **Jupyter Lab / Visual Studio Code**: Jupyter Lab is an open-source web-based interactive development environment that allowed us to quickly test and experiment

with different aspects of the pipeline, from data preprocessing to model evaluation. Visual Studio Code (VS Code) was used for the final implementation, as it supports version control, debugging, and integrates well with Python environments, making it ideal for large-scale development. Together, these tools provided a seamless workflow for both iterative experimentation and final model deployment.

3.2.4 Annotation and Dataset Handling

Effective annotation and management of training data are crucial for ensuring the quality and scalability of the model. Given the vast number of image tiles and the necessity of precise annotations, custom scripts and tools were developed to streamline the process.

- **Custom Python Scripts**: Custom scripts were written to automate the generation of datasets, the conversion of model outputs into YOLO annotation format, and the management of metadata. These scripts helped standardize the data pipeline, ensuring that every tile was processed consistently, and that metadata—such as bounding box coordinates and tile origin—was tracked and used during model training and inference.
    - Key Functions of Custom Scripts:
        - Dataset generation, including pseudo-labeling and tile extraction.
        - Metadata management, ensuring that each detection was accurately tracked.
        - YOLO format conversion for consistency across training datasets.
- **Pandas & NumPy**: **Pandas** was used for handling large tabular datasets, especially when working with metadata (such as tile indices, bounding box locations, and model confidence scores). The ability to efficiently filter, sort, and manipulate these datasets allowed us to prepare data for model training and validation. **NumPy** was used for numerical operations, such as calculating bounding box dimensions, which were essential during post-processing.
    - Key Features of Pandas & NumPy:
        - Data handling and manipulation for large datasets.
        - Efficient numerical computations on large arrays or matrices.

3.4 Summary

This chapter presented a comprehensive and technical overview of our lunar crater detection system. By combining geospatial rigor with modern deep learning techniques, we created a pipeline that is not only scientifically sound but also scalable and modular. From georeferenced tiling to pseudo-labeled annotations and accurate shapefile generation, every stage of the pipeline ensures high spatial fidelity. Terms like **georeferencing**, **semi-supervised learning**, and **selenographic inference** have been practically applied to solve a real-world planetary science problem. The result is a robust framework for crater detection—capable of accelerating lunar research and enabling accurate geological mapping for future lunar missions.

CHAPTER 4

METHODOLOGY

4.1 Overview

This chapter details the end-to-end workflow used in our crater detection pipeline. Each stage has been carefully designed to ensure spatial accuracy, efficient model training, and integration with geospatial formats. The methodology begins with acquiring calibrated OHRC imagery and ends with generating shapefiles of crater detections in selenographic coordinates. We adopt a semi-supervised approach for annotation, leveraging a small labeled dataset to bootstrap a larger one.

4.2 Data Preparation

4.2.1 Calibrated OHRC Imagery

The input data for this project consists of calibrated .img files from the Orbiter High Resolution Camera (OHRC) onboard India's Chandrayaan-2 mission. Unlike raw .img files, which are direct outputs from the imaging sensor, calibrated images are pre-processed for:

- **Radiometric Correction**: Removal of sensor noise and adjustment of brightness/contrast levels.
- **Geometric Correction**: Alignment with known control points for accurate spatial referencing.
- **Standardization**: Uniformity across different scenes and sessions.

Using calibrated data ensures that the imagery is reliable for scientific analysis and spatial mapping. Crater features appear more distinct and less prone to distortion, which is essential for high-accuracy object detection.

We intentionally avoid using raw imagery due to its lack of spatial referencing and susceptibility to artifacts, which can negatively impact model performance and coordinate transformations.

4.2.2 GeoTIFF Conversion and Georeferencing

The .img files obtained from OHRC are accompanied by .xml metadata files containing essential information such as the spatial reference system, pixel resolution, and affine transformation. These are used to convert the images into spatially-aware GeoTIFF files using GDAL. This process includes:

•. Extracting and applying the correct coordinate reference system

•. Preserving pixel scaling and orientation

•. Ensuring geolocation properties are carried into the GeoTIFF format

Next, Ground Control Points (GCPs) provided in CSV format are applied to further improve spatial accuracy. These GCPs contain precise longitude, latitude, and image coordinates to refine georeferencing.

All outputs are reprojected into a consistent lunar spatial reference system to ensure downstream detections are mappable and comparable across scenes.

4.2.3 Image Tiling and Metadata Generation

To prepare the data for YOLOv8 model training, large GeoTIFF images are divided into **512 × 512 pixel** tiles using the rasterio library. Each tile retains spatial context through:

• Its **affine transformation matrix**, which allows bounding boxes to be later mapped back to real-world coordinates

• Its **spatial extent**, i.e., the bounding box of the tile in lunar coordinates (min/max longitude and latitude)

•. A master metadata file (tile_metadata_corrected.json) that records this spatial information for all tiles

Tiles are saved in both .tif and .png formats. The .png version is used for training due to its compatibility with YOLOv8.

This tiling step enables efficient training, model inference, and later geospatial post-processing.

4.3 Annotation Strategy (Manual + Auto)

We adopted a semi-supervised annotation approach to generate crater labels for YOLOv8 training:

1. **Manual Annotation**: A small portion of image tiles was manually labeled using bounding boxes around visible craters.
2. **Initial Training**: The base model was trained using a publicly available crater dataset of 576 labeled images.
3. **Auto-Annotation**: The trained model was then used to predict bounding boxes on ~3600 unlabeled tiles.
4. **Fine-Tuning**: The model was retrained on the auto-labeled tiles to improve accuracy and generalization.

All annotations were stored in the **YOLO format**, where each bounding box is defined by its normalized center coordinates, width, and height with respect to the tile image. This format is optimized for YOLO training pipelines.

4.4 Model Selection and Training (YOLOv8)

We selected **YOLOv8** for its real-time object detection capabilities and support for custom training. YOLOv8 was fine-tuned using our annotated dataset, and key hyperparameters (batch size, learning rate, number of epochs) were optimized for our crater detection task.

The training process included:

- GPU-accelerated training on **Google Colab**
- Model monitoring using metrics like precision, recall, and IoU
- Data augmentation to improve generalization

The model was able to robustly learn spatial and textural cues associated with craters from the OHRC imagery.

4.5 Post-Processing and Geo-Mapping

Once YOLOv8 inference was complete, we used each tile's metadata to project detections into lunar coordinate space:

1. **Coordinate Transformation**: Each YOLO bounding box (in pixel coordinates) was mapped to its absolute position using the tile's affine matrix.

2. **Selenographic Adjustment**: These coordinates were translated into the lunar geographic coordinate system (selenographic latitude and longitude) to reflect real-world positioning.

This transformation ensured that detections could be accurately plotted and analyzed on the lunar surface.

4.6 Shapefile Generation

The final step was to convert detections into a shapefile format using the geopandas library. Each crater detection in the shapefile includes:

- **Crater ID**: A unique identifier
- **Selenographic Coordinates**: The projected position and size of the crater
- **Confidence Score**: The model's certainty for each detection

The shapefile enables easy integration with GIS platforms for visualization and further scientific analysis.

Table 4.1: Summary of the Crater Detection Pipeline

Stage	Description	Tools/Techniques Used	Output
Calibrated Imagery	Used radiometrically and geometrically corrected OHRC images for accuracy	ISRO OHRC dataset	Calibrated .img + .xml files
GeoTIFF Conversion	Converted .img to GeoTIFF using metadata and GCPs for spatial referencing	GDAL, CSV Ground Control Points	Georeferenced GeoTIFFs
Image Tiling	Split images into 512×512 tiles while preserving spatial metadata	Rasterio	Tiles in .png and .tif + metadata JSON
Semi-Supervised Annotation	Bootstrapped labels using a small manually labeled dataset and auto-labeled the rest	Roboflow, YOLOv8, Custom Labeling	Annotated dataset in YOLO format
Model Training (YOLOv8)	Trained YOLOv8 with tuned hyperparameters and data augmentation	Google Colab, YOLOv8	Trained crater detection model
Inference and Mapping	Mapped YOLO detections to lunar selenographic coordinates using affine transformation	NumPy, Rasterio, Tile Metadata	Crater detections in lunar coordinates
Shapefile Generation	Converted detections into GIS-compatible shapefile format	GeoPandas	Shapefile with Crater ID, Coordinates, Scores

Conclusion of Chapter 4

This chapter outlined the complete methodology for detecting and georeferencing lunar craters using calibrated OHRC imagery, a YOLOv8-based detection pipeline, and geospatial processing tools. The pipeline is highly modular, scalable, and spatially accurate, enabling reproducible crater detection and mapping on the lunar surface.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Model Performance Metrics

After training the YOLOv8 model for 50 epochs, we evaluated its detection performance on the test set of lunar tiles. The metrics at the final epoch were:

- Precision: 88.2%
- Recall: 79.6%
- F1-Score: 83.6%

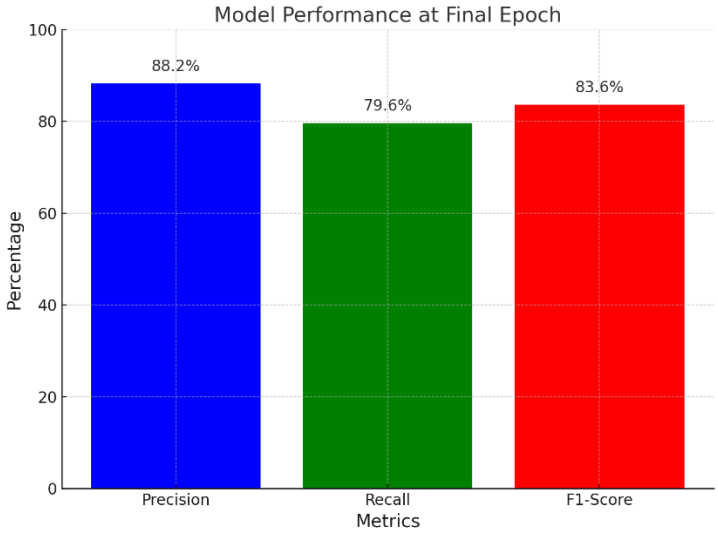


Figure 5.1:Model performace at Final Epoch

These metrics were derived from the best epoch of training and reflect a strong balance between minimizing false positives and capturing true crater instances.

The evolution of these metrics over time is shown in the following performance curves:

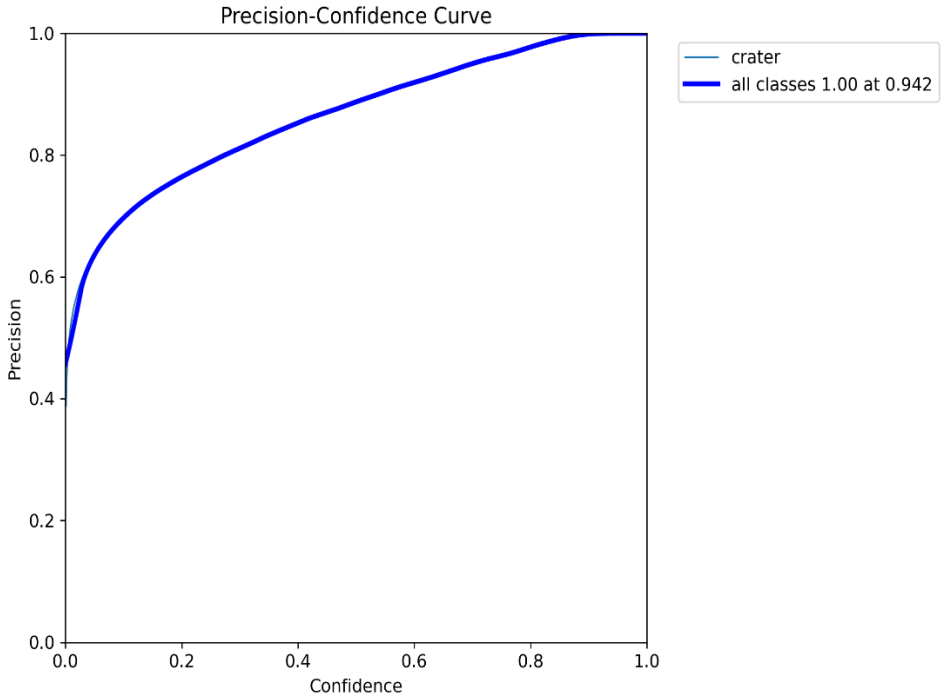


Figure 5.2: Precision Curve

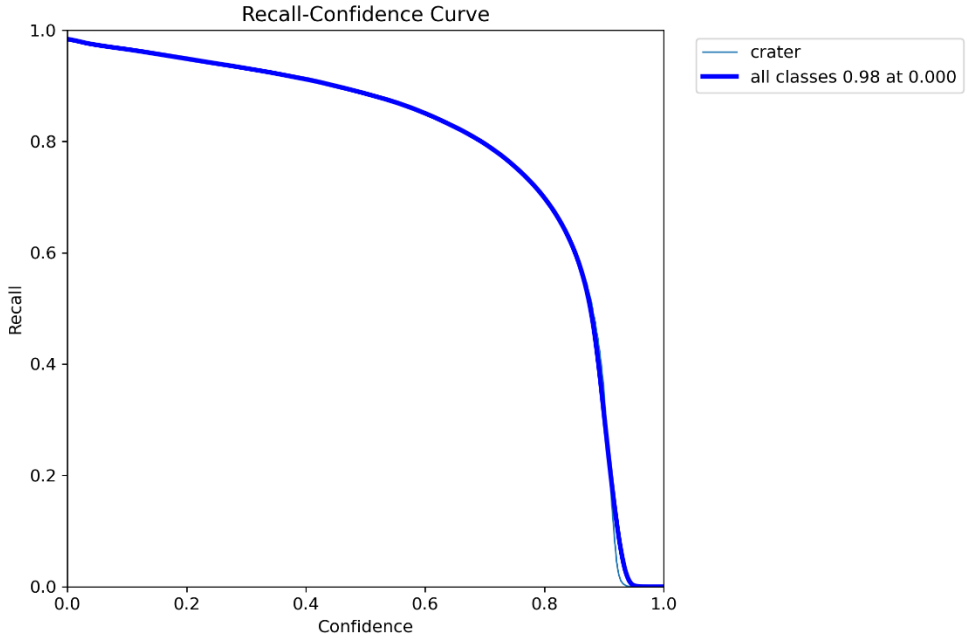


Figure 5.3: Recall Curve

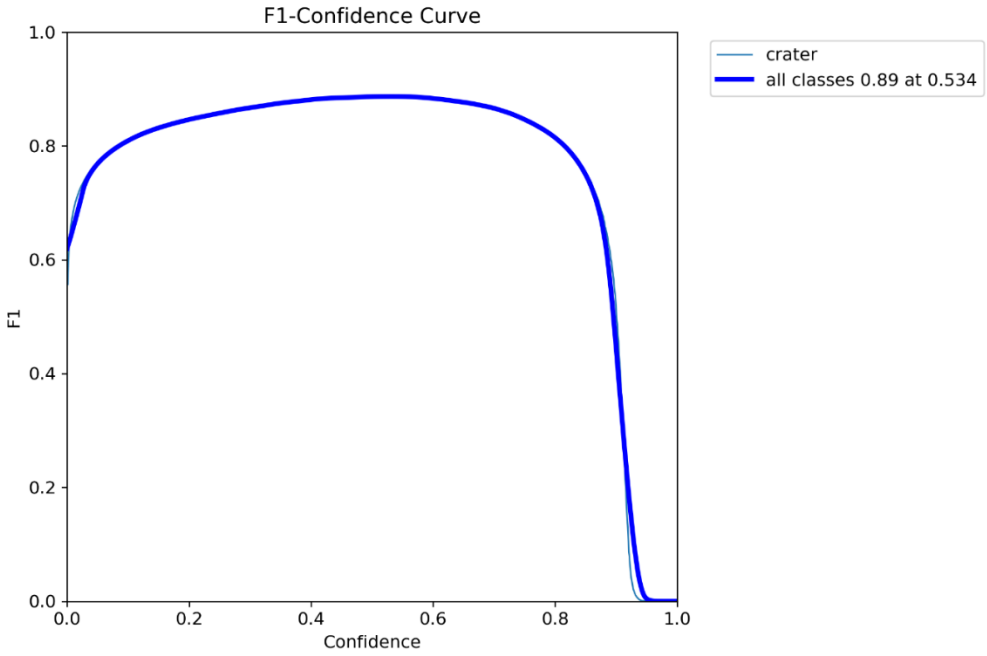


Figure 5.4: F1-Score Curve

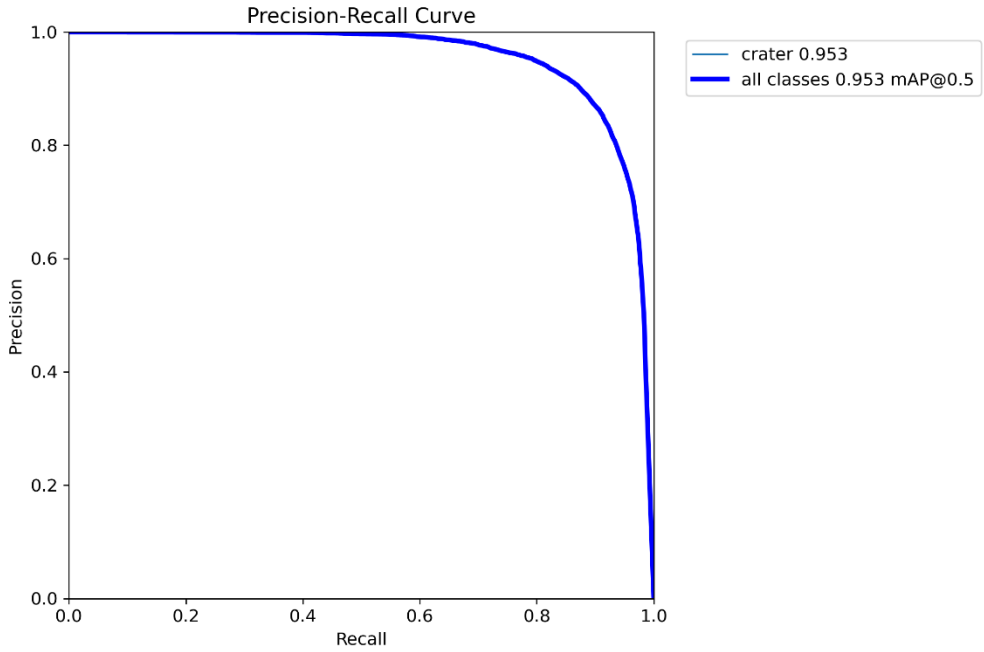


Figure 5.5: Precision-Recall (PR) Curve

Each curve demonstrates a consistent improvement and convergence of model performance across training epochs.

5.2 Confusion Matrix and Class-Specific Insights

Since our dataset was single-class (crater detection), the confusion matrix appears simplistic but is still informative in verifying that the model is not confused by background noise or other terrain features.

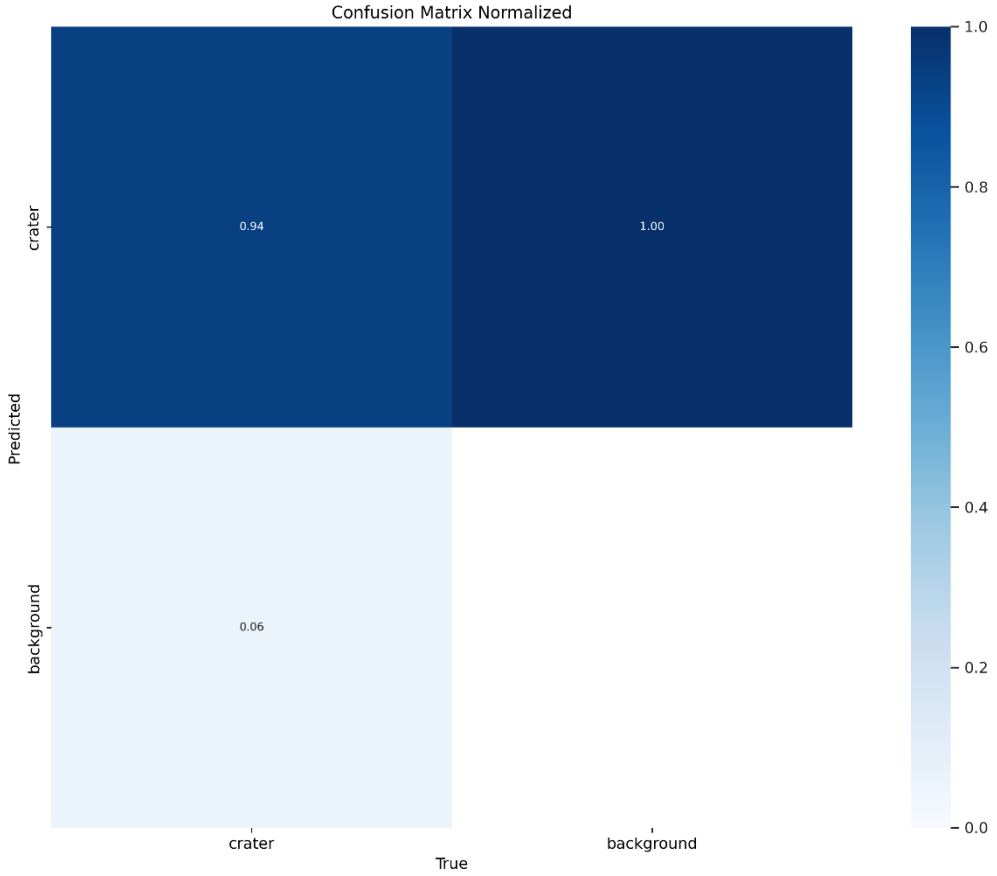


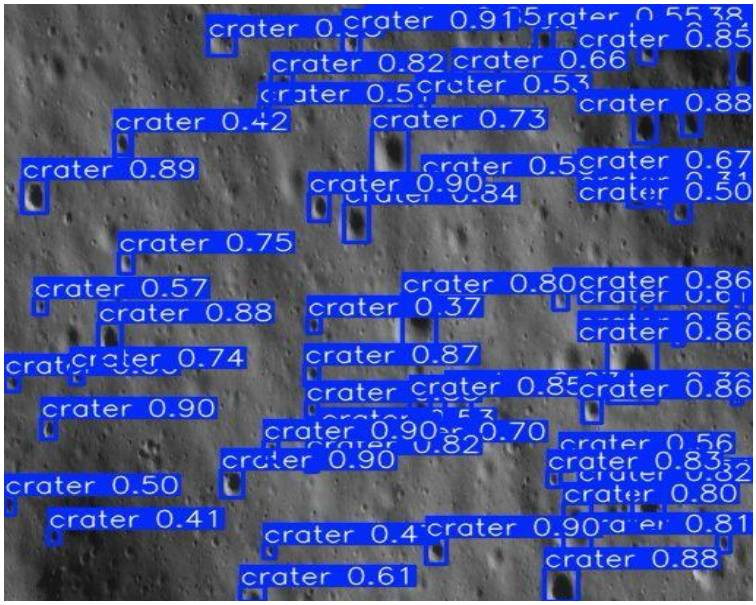
Figure 5.6: Normalized Confusion Matrix

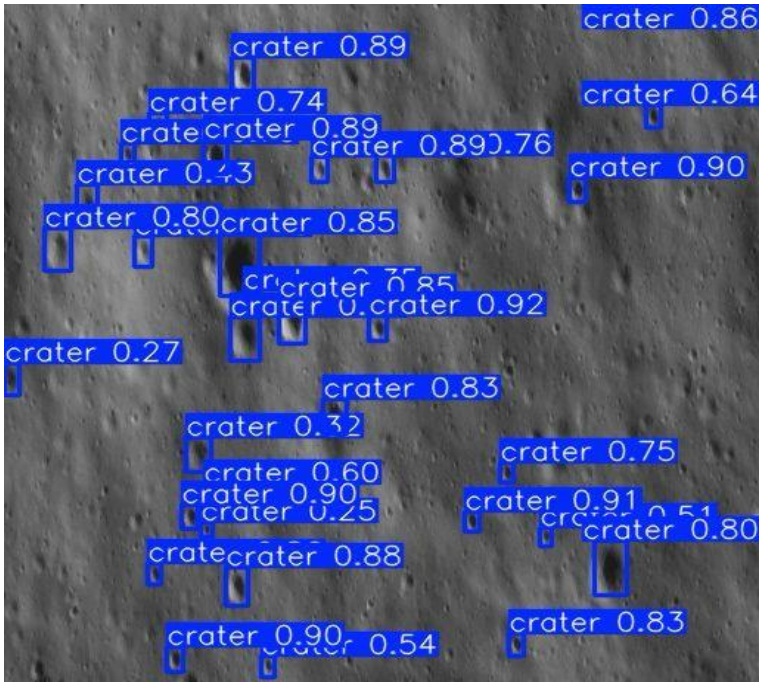
The normalized matrix shows that ~79% of the ground truth crater annotations were correctly predicted as craters, while the remaining ~21% were missed detections (false negatives). No significant false positives were observed, which validates the model's specialized training.

Although the confusion matrix appears limited in its visual impact due to single-class training, it still affirms that the model has correctly learned the crater features and avoids overgeneralization.

5.3 Qualitative Results on Lunar Tiles

Visual inspection of inference results is critical in a domain like lunar surface analysis, where terrain variations and lighting conditions can mislead even high-performing models





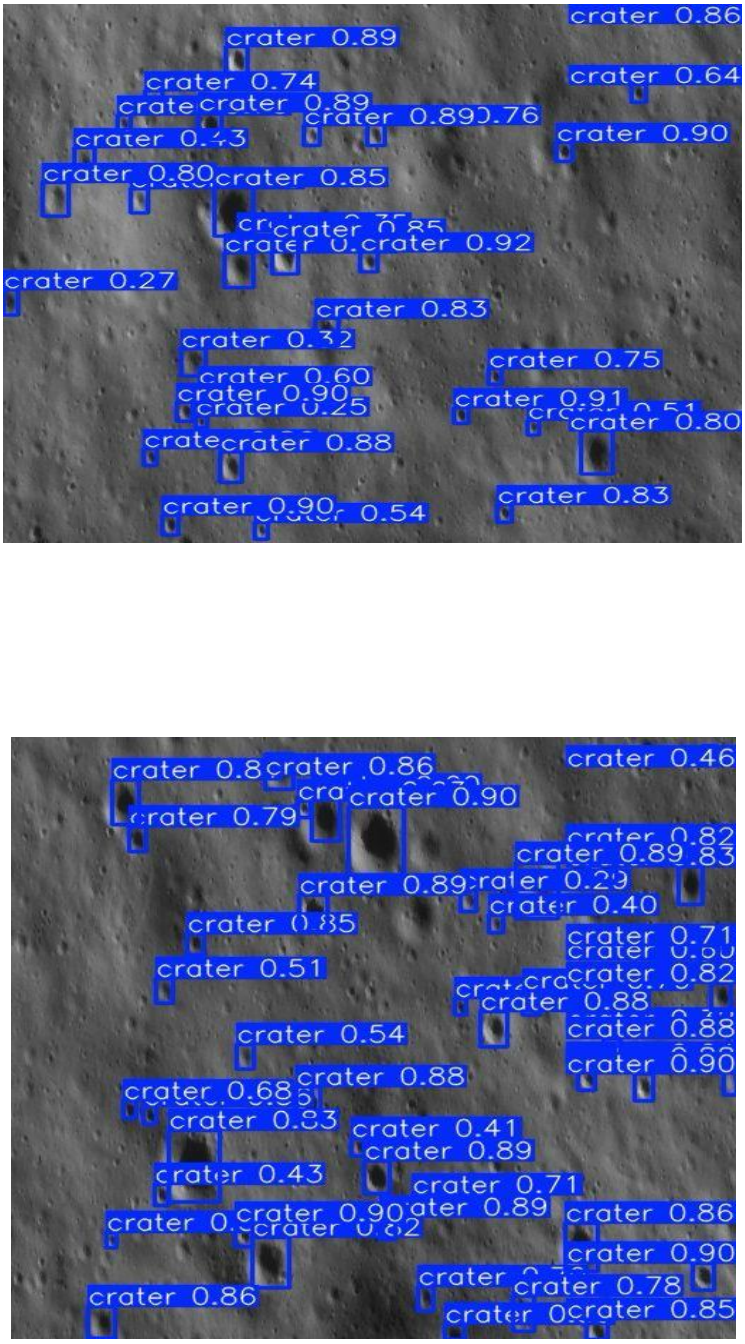


Figure 5.7: Sample Detection Outputs (from test set)

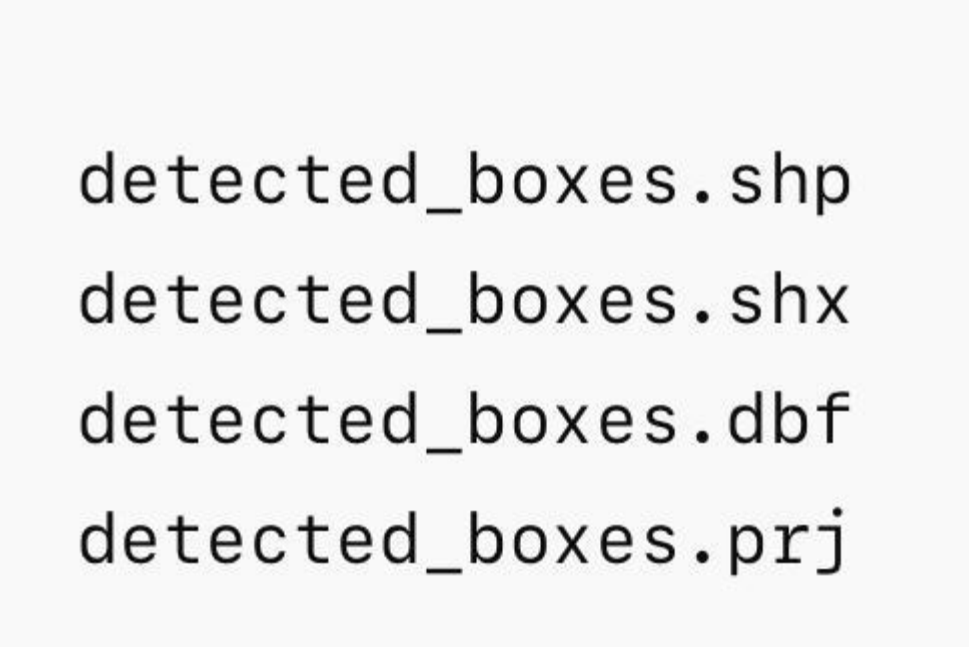


Figure 5.8: Generated Shapefiles

The model correctly detects craters of various diameters and shapes, especially those with clear rims and contrast. In some cases, the model also picks up degraded or shadowed craters, indicating its robustness.

The bounding boxes are tight and confidence scores remain consistently high, especially for medium to large craters. Smaller or partially visible craters — particularly those at tile boundaries — pose more of a challenge.

5.4 Observations and Inference Notes

- The semi-supervised annotation approach enabled us to scale our dataset from 576 manually labeled images to over 3600 annotated tiles, drastically improving model generalization.
- Training was conducted on Google Colab using GPU acceleration, while inference and post-processing were performed on a macOS system.
- The model demonstrates robustness across tile-to-tile brightness and contrast variation — a common challenge in lunar imagery.
- Using calibrated imagery helped reduce artifacts and noise, improving both annotation quality and model accuracy.

- The consistent detection of crater rims and shadows indicates that the model learned relevant spatial and textural features.

CHAPTER 6

CHALLENGES AND LIMITATIONS

6.1 Technical Challenges

Throughout the development of this lunar crater detection system, we encountered various technical challenges at different stages of the pipeline. Some of the key hurdles are outlined below:

- Georeferencing Accuracy

Converting .img files to georeferenced GeoTIFFs while preserving spatial fidelity required careful application of .xml metadata and ground control points (GCPs). Even small errors in affine transformation could affect crater mapping precision.

- Metadata Handling

Extracting projection and transformation information from the .xml files and aligning them with the GCPs demanded consistent checks to ensure spatial correctness throughout the pipeline.

- Annotation Pipeline Setup

Setting up the semi-supervised annotation strategy—using a model trained on a small open-source dataset to auto-label 3600+ tiles—required iterative quality checks and re-annotation cycles. Model confidence thresholds also had to be tuned to reduce false positives.

- Model Training Infrastructure

Although model training was successfully conducted using Google Colab with GPU acceleration, care had to be taken in managing session timeouts, storage constraints, and dependency setups in each session. Inference was later run on a macOS system using a local CPU-only setup, which, while sufficient for demonstrations, is not ideal for large-scale processing.

6.2 Data-Related Challenges

- Limited Labeled Datasets

The lack of large-scale, accurately labeled crater datasets led us to adopt a semi-supervised annotation approach. While effective, this introduced noise and required fine-tuning of the model using auto-labeled data.

- Visual Similarity Between Craters and Terrain

In some tiles, crater rims and shadows were difficult to distinguish from surrounding lunar features such as small hills or rough textures, occasionally leading to incorrect detections.

- Tile Boundary Effects

Craters at the edges of tiles sometimes appeared incomplete or fragmented, leading to either duplicate detections across tiles or false negatives during prediction.

6.3 Known Limitations

Despite the successful implementation and promising results, the current system has certain limitations:

1. Single-Class Detection

The model has been trained exclusively for crater detection. It does not differentiate between crater types or other lunar features such as boulders or rilles.

2. Dependence on Calibration Quality

The accuracy of detection and spatial mapping is dependent on the quality of the pre-calibrated OHRC imagery. Any upstream errors in calibration may propagate through the pipeline.

3. Edge Artifacts in Tiling

Detections near tile boundaries may suffer from truncation or duplication, as the model lacks context from adjacent tiles.

4. Auto-Annotation Accuracy

The semi-supervised annotation approach relies on the assumption that the initial model generalizes well. While useful for bootstrapping, this can introduce label noise, especially in visually ambiguous regions.

5. Not Yet Optimized for Real-Time Deployment

While the crater detection pipeline shows potential for real-time or near-real-time applications—such as onboard satellite processing or lunar mission planning tools—the current implementation is optimized for offline batch processing.

Future versions may explore deployment optimizations such as model pruning, quantization, lightweight YOLO variants, or hardware acceleration to enable real-time use.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project successfully developed a semi-automated system for detecting and mapping lunar craters using calibrated OHRC imagery and a YOLOv8-based deep learning model. Starting from georeferenced .img data, the complete workflow encompassed image tiling, model training, inference, and post-processing to generate accurate crater detections, which were subsequently mapped to selenographic coordinates through shapefile generation.

By adopting a semi-supervised approach—bootstrapping a small manually labeled dataset to annotate over 3600 additional tiles—we were able to fine-tune a robust model that generalizes well across varied lunar terrains. The final model achieved a precision of 88%, with high-quality detection outputs and spatially coherent shapefiles, positioning this system as a valuable tool for lunar geological analysis and mission planning.

The seamless integration of geospatial processing with machine learning techniques proved vital in managing lunar remote sensing datasets and lays the groundwork for future lunar surface exploration technologies.

7.2 Future Enhancements

To improve the current system and broaden its applicability, several future directions are proposed:

1. Multi-Class Detection

Extend the model to detect additional lunar surface features—such as boulders, rilles, or lava tubes—using class-specific annotations and training strategies.

2. Real-Time Inference Pipeline

Optimize the system for real-time or near-real-time use on edge devices, including potential deployment onboard lunar missions, by implementing model pruning, quantization, or conversion using frameworks like TensorRT.

3. Context-Aware Tiling

Introduce overlap-aware tiling strategies and post-processing logic to mitigate edge artifacts and ensure seamless detections across tile boundaries.

4. Improved Auto-Annotation Strategy

Incorporate uncertainty estimation or active learning to refine noisy auto-labeled data, enhancing annotation quality and overall model reliability.

5. Interactive Visualization Tool

Develop a lightweight GIS-based frontend to interactively visualize shapefiles, inspect detection results, and overlay relevant metadata, aiding scientific review and validation.

6. Higher Resolution and Multi-Scale Training

Train models on higher-resolution tiles or with multi-scale data augmentation to boost detection performance on small craters and improve boundary localization.

REFERENCES

1. https://www.researchgate.net/publication/384030307_A_Novel_Deep_Learning_Approach_for_data_analysis_on_high_resolution_Chandrayaan-2_Data
2. https://education.nationalgeographic.org/resource/crater/
3. http://www.isro.gov.in
4. https://www.qgis.org/en/site/
5. https://roboflow.com/annotate
6. https://universe.roboflow.com/college-702m5/craters-clxwc
7. https://doi.org/10.3390/rs15205040
8. https://pradan.issdc.gov.in/ch2/
9. https://yolov8.org/how-to-use-yolov8-for-object-detection-2/
10. Y. Jia, G. Wan, L. Liu, Y. Wu and C. Zhang, "Automated Detection of Lunar Craters Using Deep Learning," 2020 IEEE 9th Joint International Information Technology and Artificial Intelligence Conference (ITAIC), Chongqing, China, 2020, pp. 1419-1423, doi: 10.1109/ITAIC49862.2020.9339179.
11. Sinha, M., Paul, S., Ghosh, M. et al. Automated Lunar Crater Identification with Chandrayaan-2 TMC-2 Images using Deep Convolutional Neural Networks. Sci Rep 14, 8231 (2024). https://doi.org/10.1038/s41598-024-58438-4
12. Automatic Mapping of Small Lunar Impact Craters Using LRO-NAC Images,Earth and Space Science,https://doi.org/10.1029/2021EA002177
13. https://www.researchgate.net/publication/360121927_Review_for_Examining_the_Oxidation_Process_of_the_Moon_Using_Generative_Adversarial_Networks_Focusing_on_Landscape_of_Moon